



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W3D5 HTTP Basic 认证与受保护路由

WebServer V1.5: 在 V1.4 路由前增加认证

从 V1.4 到 V1.5：部分路由需要身份验证

V1.4 已完成配置加载和 method + path 路由，本次只增加一层认证检查。

- GET /: 公开路由，任何客户端都可访问。
- GET /secured: 受保护路由，Basic 凭据正确才执行 handler。
- 认证发生在路由匹配之后、handler 之前。
- epoll 事件循环和 HTTP 请求解析保持不变。

V1.5 的核心：同一台服务器，不同路由可以采用不同安全策略。



先区分认证与授权

认证 Authentication：确认“你是谁”。

- 没有凭据或凭据错误 -> 401 Unauthorized。
- 401 响应告诉客户端应采用哪种认证方案。

授权 Authorization：确认“你能做什么”。

- 身份已确认，但无权访问资源 -> 403 Forbidden。

路由不存在 -> 404 Not Found，与身份是否正确无关。

本实验只实现认证，不实现角色或权限控制。



三种方案解决的不是同一个问题

Basic

- 每次请求发送 Base64(username:password)，服务端校验凭据。

Session

- 客户端携带随机 SessionID；服务端保存并查询会话状态。

Bearer Token

- 客户端发送 Authorization: Bearer <token>。Token 可以是不透明值，也可以是 JWT。
- Token 不等于 JWT，也不一定完全不查询服务端。

共同要求：生产环境必须使用 HTTPS。本次必做只实现 Basic。



Basic 认证是一次 401 挑战

Basic 不是登录一次后保存会话；客户端访问受保护路由时，每次请求都携带 Authorization。

- 1 客户端 -> GET /secured (无 Authorization)
- 2 服务端 <- HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="mini_webserver"
- 3 客户端 -> GET /secured
Authorization: Basic <Base64(username:password)>
- 4 服务端 <- 凭据正确：200 OK
凭据错误：401 Unauthorized (不是 403)



解析 Authorization 后再决定状态

服务器只对 auth=basic 的路由执行以下检查：

1. 读取唯一的 Authorization 请求头。
 2. 检查 scheme 是否为 Basic，并提取编码字符串。
 3. Base64 解码，按第一个冒号拆分 username:password。
 4. 与服务器保存的凭据进行比较。
- 缺少、scheme 错误、Base64 无效、缺少冒号或密码错误 -> 401。
 - Authorization 重复或超过请求头长度限制 -> 400。
 - 凭据正确 -> 继续执行当前路由的 handler。
 - 日志不得输出 Authorization、Base64 字符串或密码。



认证中间件位于路由和 handler 之间

完整 HTTP 请求

|

根据 method + path 查找路由

|-- 未找到 -----> 404 / 405

|

+-- 找到 route -> route.auth == "basic" ?

| 否 -> handler(req)

| 是 -> validate_basic(req)

|-- 通过 -> handler(req)

+-- 失败 -> 401 + WWW-Authenticate

配置示例: {"method":"GET", "path":"/secured", "handler":"secured_get", "auth":"basic"}



Basic 认证只能在安全边界内使用

- Base64 是编码，不是加密；抓到请求即可还原用户名和密码。
- 生产环境必须使用 HTTPS，防止传输过程被窃听或篡改。
- 密码不能硬编码在源代码中，也不能写入访问日志。
- 真实系统保存密码哈希；验证时避免泄露比较结果和时序信息。
- 受保护响应建议添加 Cache-Control: no-store。
- 限制请求头大小，认证失败可配合限速，降低暴力尝试风险。

实验边界：只在 localhost 演示协议流程，不把示例凭据用于真实系统。

训练内容：实现 HTTP Basic 认证

在已经完成的 WebServer V1.4 中加入 HTTP Basic 认证，实现以下 3 个访问场景：

1. 访问公开页面

访问：http://localhost:8080/

结果：HTTP/1.1 200 OK，可以正常查看首页。

2. 未携带认证信息访问受保护页面

访问：http://localhost:8080/secured

结果：HTTP/1.1 401 Unauthorized，并返回 WWW-Authenticate 响应头。

3. 使用正确账号和密码访问受保护页面

账号：student 密码：lab123

访问：http://localhost:8080/secured

结果：HTTP/1.1 200 OK，并返回受保护页面内容。

补充要求：账号或密码错误仍返回 401；V1.4 原有功能继续可用；日志不得输出认证凭据。

最低验收证据

依次执行：

```
BASE=http://127.0.0.1:9090
```

```
curl -i "$BASE/"
```

```
curl -i "$BASE/secured"
```

```
curl -i -u student:wrong "$BASE/secured"
```

```
curl -i -u student:lab123 "$BASE/secured"
```

预期结果：

- / 返回 200；无凭据返回 401 且包含 WWW-Authenticate。
- 错误凭据返回 401；正确凭据返回 200。
- 日志中没有 Authorization、Base64 凭据或密码。
- V1.4 的静态资源、404/405、配置和 fd 清理测试继续通过。



扩展训练：Session 与 Bearer Token

选做 1：Session

- 使用不可预测的随机 SessionID；登录后重新生成，退出和过期时在服务端失效。
- Cookie 设置 HttpOnly、Secure、SameSite，并增加 CSRF 防护。
- SessionID 被窃取属于会话劫持；CSRF 的根因是浏览器自动携带 Cookie。

选做 2：Bearer Token / JWT

- 使用 Authorization: Bearer <token> 和成熟认证库。
- JWT 必须验证签名、exp、iss、aud；负载可读，不能保存密码。
- 不透明 Token 仍可能需要查询服务端状态。

两项均不替代 Basic 必做，也不允许自行实现加密算法。

